

문제가 보이면 **손이 먼저 움직이는** 개발자 김서현입니다.

약 2년간 여러 차례의 팀 프로젝트 및 실무형 협업을 통해, 빠른 프로토타이핑으로 기획 의도를 검증하고 동시다발적인 기술 이슈(동시성 제어, 프론트 상태 관리 최적화 등)를 주도적으로 해결해 왔습니다. 또한, 필요한 순간에는 부족한 포지션(프론트엔드/백엔드/인프라/설계)을 스스로 학습하고 투입되어 제품 완성도를 책임져왔습니다.

개발 영역 바깥에서도 마찬가지입니다. Jira, Notion, Figma 기반의 문서화와 일정 관리를 주도하여 팀 내 의사소통 비용을 줄였고, GitLab, Jenkins, Docker기반의 CI 자동화 파이프라인을 직접 설계 · 운영해보았습니다. 그 밖에 GitLab Webhook과 Jira를 연동하여 이슈 상태 자동 전환이나 팀의 커밋 규칙을 반영한 ChatGPT 커밋 자동 생성기를 만들어 팀 내에서 사용했습니다.

말보다는 눈에 보이는 결과물로 대화하고, 무조건적으로 최신/최상의 기술을 쫓기 보다는 주어진 환경에서 최적의 기술을 찾아 적용합니다. 더 좋은 시스템, 더 좋은 팀을 만드는 일이라면 어디서든 최선을 다할 자신이 있습니다.

주요 기술스택



객체지향 설계 원칙을 기반으로 유지보수하기 좋은 백엔드 구조 설계 가능



컴포넌트 설계, 상태 관리, API 연동까지 실서비스 수준의 웹 프론트엔드 개발 가능



REST API 설계부터 운영 환경 배포까지, 백엔드 전 과정을 주도적으로 진행한 경험



Jetpack Compose로 선언형 UI 구성, 네이티브 앱 처음부터 끝까지 구현 가능



전역 상태 구조를 목적에 맞게 설계하고, 비동기 흐름과 함께 효율적으로 관리 가능



요구사항에 맞는 스키마 설계, 인덱스 전략 수립, 복잡한 조인 쿼리 작성 가능



메모리 구조와 포인터 이해, 저수준 프로그래밍 및 알고리즘 구현 경험



API 연동 및 인터셉터를 활용한 공통 인증 처리, 에러 핸들링 구조 설계 경험



기본적인 엔티티 설계와 연관관계 매핑을 활용한 CRUD 구현 경험



seohyeon.shkim@gmail.com



<https://github.com/seohye-ki>



<https://seohye-ki.github.io/>

교육

삼성 청년 SW · AI 아카데미

2024.07 ~ 2025.06

삼성전자와 고용노동부가 주관하는 실무형 SW 교육과정으로, 팀 프로젝트를 통해 기획부터 배포까지 경험할 수 있도록 설계된 교육 과정

- Java, Spring Boot, Vue.js, React, MySQL을 활용한 프로젝트 수행
- 알고리즘 모의 A형 취득
- 우수 수료자(상위 30%)

42 서울

2023.08 ~ 2024.07

정해진 강의나 교재, 교수 없이 전적으로 자기주도 학습과 동료 평가를 통해 성장하는 교육 과정

- C, C++, 시스템 프로그래밍, 알고리즘, 네트워크 등 CS 학습

서울여자대학교

2018.03 ~ 2023.02

경영학과(본전공) / 디지털융합경영(연계전공)

디지털융합경영전공이란?
IT와 경영학을 결합한 교육과정으로, 프로그래밍 언어(C/C++), 알고리즘, 컴퓨터 구조 등 기초 CS 지식과 데이터 분석(R/구글애널리틱스) 프로그래밍 학습

수상내역

삼성 청년 SW · AI 아카데미 자율 프로젝트(도메인: 오픈소스) 경진대회 - 우수상(3위)

AI 발표 코칭 안드로이드 애플리케이션 (Spico) 출품

2025.05.22

삼성 청년 SW · AI 아카데미 특화 프로젝트(도메인: 핀테크) 경진대회 - 우수상(2위)

신용도 기반 초등학생 금융교육 시뮬레이션 (LuFin) 출품

2025.04.11

삼성 청년 SW · AI 아카데미 공통 프로젝트(도메인: 웹디자인) 경진대회 - 우수상(3위)

IT 개발 공식 문서 공동 번역 및 커뮤니티 (DocshunD) 출품

2025.02.21

삼성 청년 SW · AI 아카데미 관통 프로젝트(도메인:스포츠) 경진대회 - 우수상(2위)

프로스포츠 경기일정 통합 캘린더 (EVERYMATCH) 출품

2024.11.28

자격증 / 어학

정보처리기사

2022.11.25

ADsP (데이터분석 준전문가)

2025.06.13

SQLD (SQL 개발자 자격증)

2024.12.13

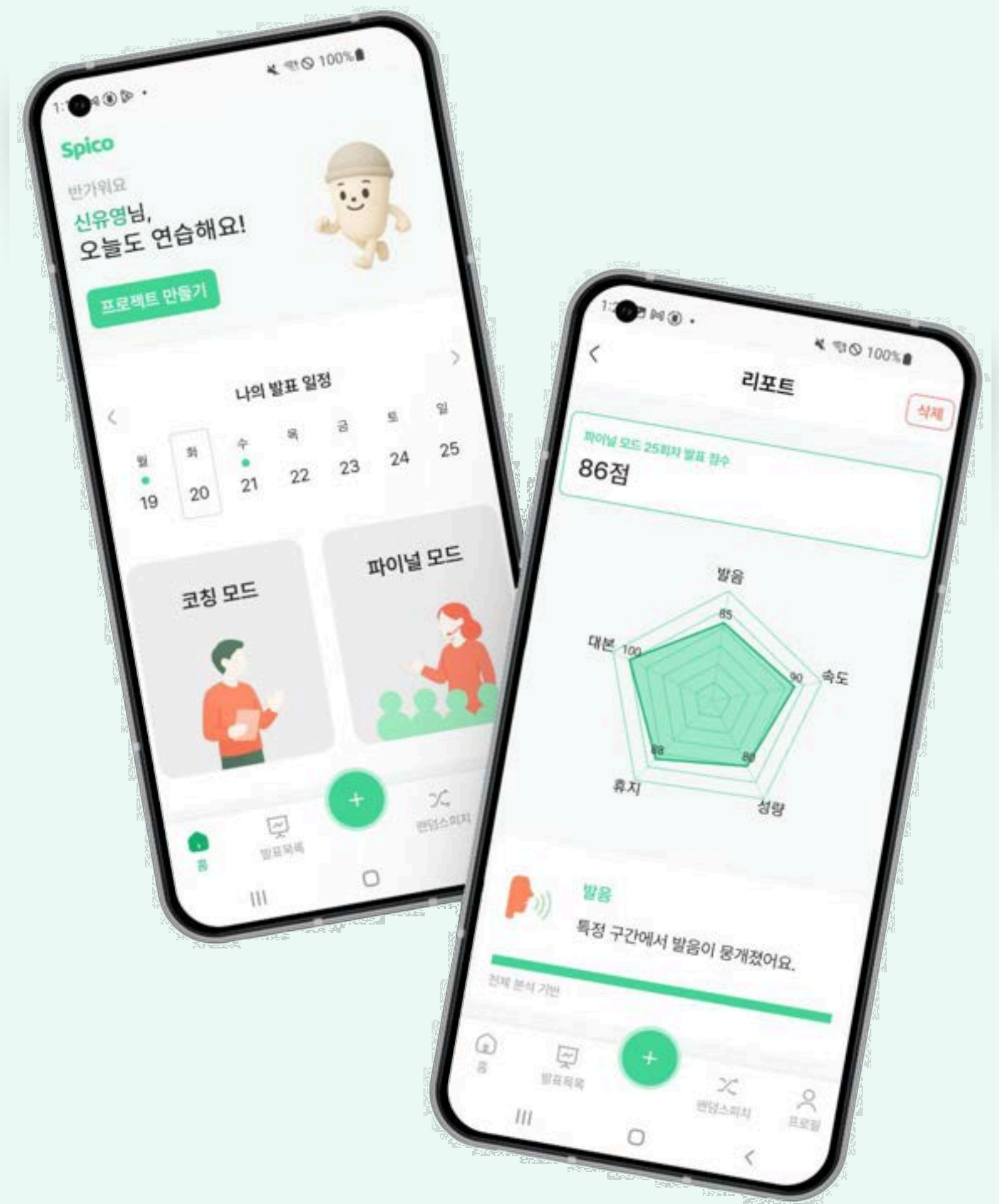
Toeic - 835

2026.01.11

Toeic Speaking - IM3 (130)

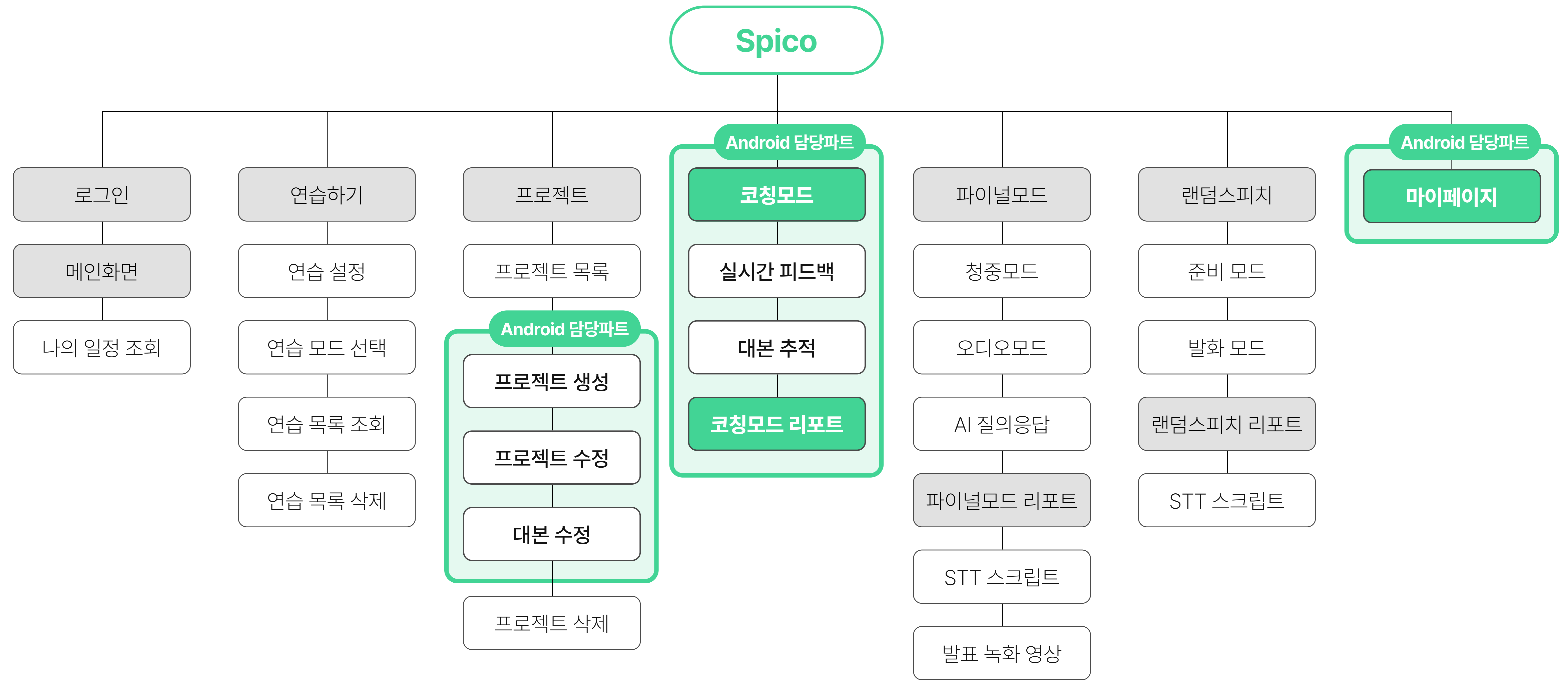
2025.09.27

프로젝트명	Spico 우수상 수상 🏆
개요	AI 기반 실시간 피드백과 발표 시뮬레이션을 제공하는 발표 연습 안드로이드 앱
개발기간	2025.04.14 ~ 2025.05.22 (6주)
개발인원	6명
담당역할	안드로이드, UI/UX디자인
결과물	Github 발표자료 영상자료
기술 환경	
안드로이드	Kotlin, Jetpack compose, Android SDK
백엔드	Spring, Kotlin, Java
데이터베이스	MariaDB
데브옵스	Jenkins, Docker, Nginx, AWS EC2, Minio, Prometheus, Grafana
협업	Jira, Gitlab, Figma, Notion
기타	JWT, GPT 4o, Azure STT, Google STT, Whisper



AI 피드백 발표 코칭 앱 |

Jetpack Compose 기반 Clean Architecture를 도입하여 유연하고 검증 가능한 코드로 안드로이드 앱을 완성했습니다.



트러블 슈팅

Problem 1

STT 기반 대본 추적 알고리즘 개선

문제상황

기존 대본 추적 기능은 STT partial 결과와 문장 전체를 Levenshtein Distance 기반 유사도로 비교하여 일정 threshold 이상일 경우 하이라이트를 이동하는 방식이었습니다. 문자열 오차에는 강했지만, 문장 전체 기준으로 계산 하다 보니 사용자가 문장의 앞부분만 읽었을 경우 유사도가 충분히 나오지 않아 문장을 거의 끝까지 말해야 하이라이트가 이동하는 문제가 있었습니다. 연습 모드 특성상 특정 문단을 바로 연습하고 싶어도 즉시 반응하지 않아 UX가 매끄럽지 못한 상황이었습니다.

해결과정

threshold 완화와 다른 유사도 알고리즘 도입을 검토했으나, 오인식 가능성 증가와 실시간 처리 부담을 고려해 적합하지 않다고 판단했습니다. 대신 알고리즘을 교체하기보다 판단 기준을 수정하는 방향을 선택했습니다. 기존의 “얼마나 정확히 말했는가” 중심 구조에서, “해당 문단을 시작했는가”를 우선 판단하도록 변경했습니다. 문장 앞부분 비교를 먼저 수행하고, Levenshtein 유사도를 활용하는 방식으로 개선하여 문장의 일부만 읽어도 빠르게 하이라이트가 이동하도록 수정했습니다.

결과 및 배운점

개선 이후 문장 초반만 읽어도 즉시 문단이 추적되도록 반응성이 향상되었고, 연습 모드 사용성이 개선되었습니다. 이번 작업을 통해 알고리즘의 정확도보다 중요한 것은 사용 맥락에 맞는 판단 기준이라는 점을 배웠으며, 새로운 기술 도입보다 문제 정의를 재설정하는 것이 더 효과적인 해결이 될 수 있음을 경험했습니다.

문단 1

안녕하세요, 오늘 저는 인공지능이 우리의 삶을 어떻게 변화시키고 있는지에 대해 이야기해보려고 합니다.

문단 2

인공지능 기술은 의료, 교육, 금융 등 다양한 산업에서 빠르게 활용되고 있습니다.

개선 전 발화

 안녕하세요, 오늘 저는 인공지능이 우리의 삶을 어떻게

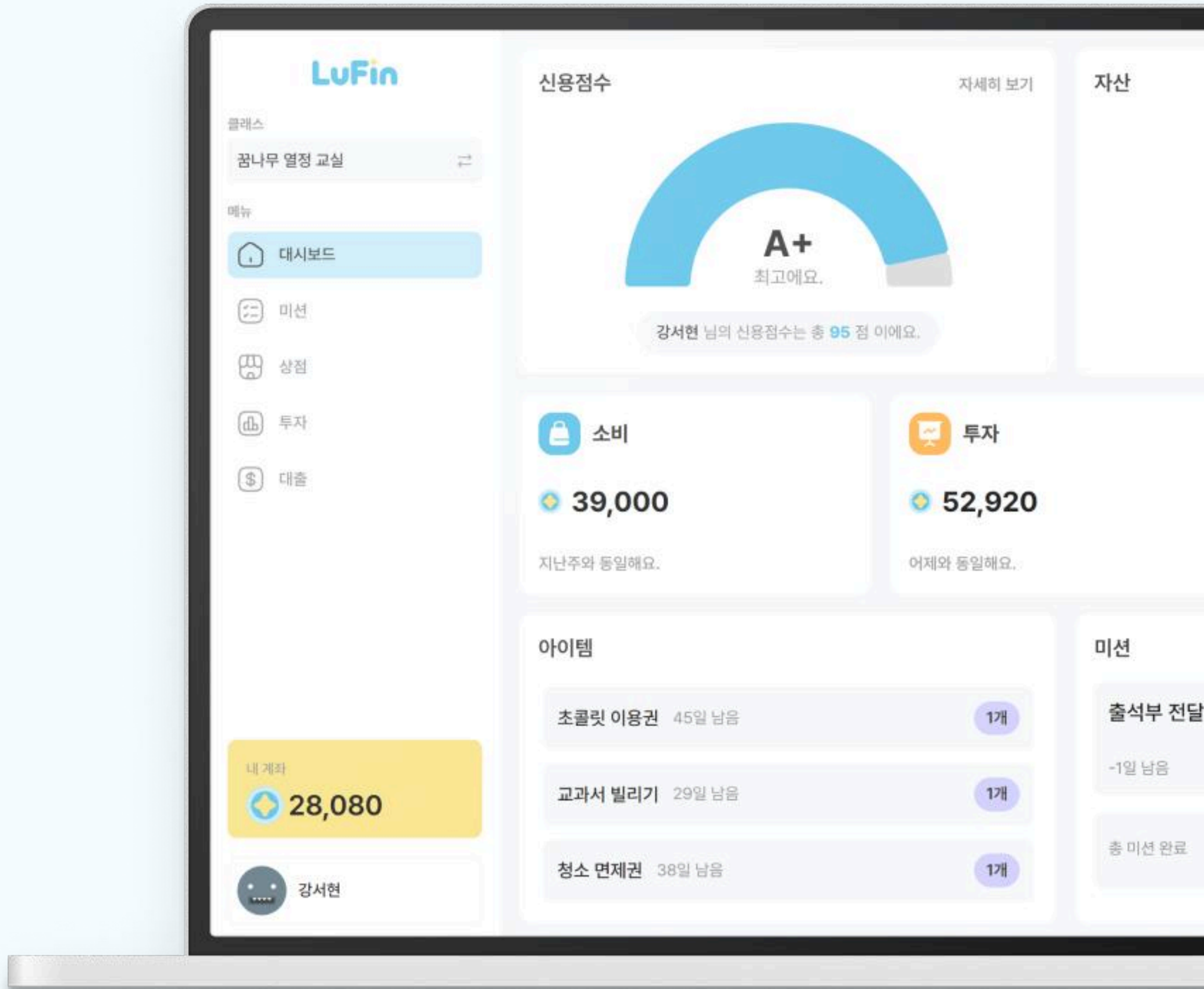
STT Text ▶ 전체 문장 비교 ▶ Levenshtein 0.4 이상 문단 선택

개선 후 발화

 안녕하세요, 오늘 저는

STT Text ▶ 문장 앞부분 비교 ▶ Levenshtein 최고 점수 문단 선택

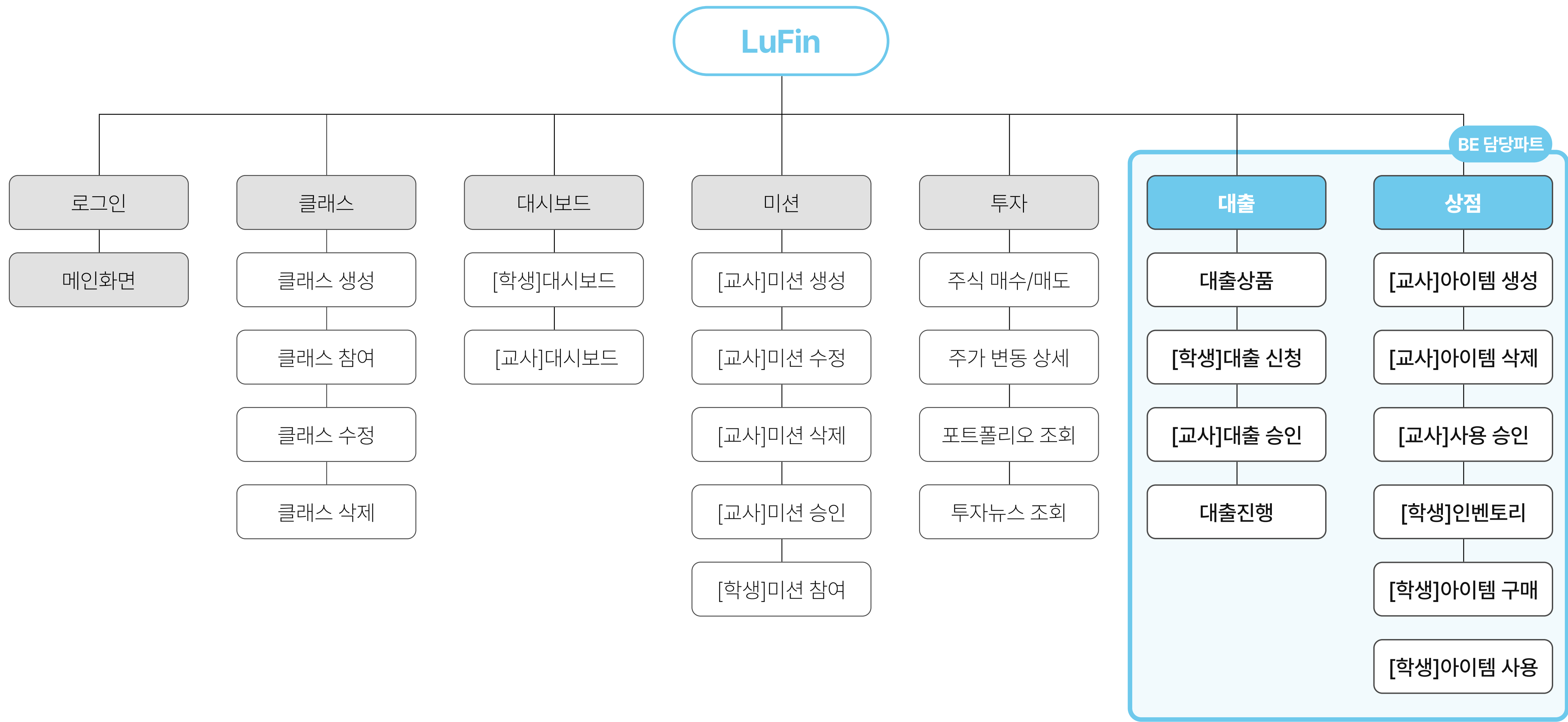
프로젝트명	LuFin 우수상 수상 🏆
개요	신용도 기반 경제활동 시뮬레이션 플랫폼, 초등학교 고학년을 대상으로 교실에서 진행되는 경제 금융 교육 활동 지원
개발기간	2025.02.24 ~ 2025.04.11 (7주)
개발인원	6명
담당역할	백엔드, UI/UX디자인, 인프라
결과물	Github 발표자료
기술 환경	
프론트엔드	React, TypeScript, Zustand, Tailwind CSS, Axios
백엔드	Spring Boot, JPA, QueryDSL
데이터베이스	MySQL, Redis
데브옵스	Jenkins, Docker, Nginx, AWS EC2, S3
협업	Jira, Gitlab, Figma, Notion
기타	JWT, Claude AI, Chart.js



신용도 기반 금융교육 |

100명이 동시에 접근하는 환경에서도 오차 없는 금융 시뮬레이션을 위해 JPA 비관적 락(Pessimistic Lock)으로 동시성을 제어했습니다.

서비스 구성도 및 담당 개발 파트



Problem 1

다중 동시 접속 환경에서의 선착순 구매 Race Condition 대응

문제상황

인기 있는 상품 아이템을 다수의 학생이 동시에 구매할 때, 재고 수량 이상의 아이템이 판매되는 초과 결제 현상이 우려되었습니다. 락(Lock)이 없는 상태에서 남은 재고가 1개일 때 100명의 학생이 동시에 구매 요청을 보내면, 스레드들이 동시에 동일한 재고(1개) 상태를 읽어들이게 됩니다. 이로 인해 트랜잭션 종료 시 재고가 마이너스로 표기되며 재고 이상의 결제 요청이 성공해버리는 데이터 정합성 파괴 시나리오가 존재하는 것을 찾아냈고, 테스트 결과 초과 결제율이 90% 이상 발생하는 것을 확인하였습니다.

해결과정

동시성 이슈를 해결하기 위해 낙관적 락, Redis 활용 분산 락, 그리고 비관적 락 등 여러 제어 방식을 검토했습니다. 아이템 구매 기능의 특성상 다수의 사용자가 동시에 접근하여 트랜잭션 충돌 연산이 매우 빈번하게 발생하여 충돌 발생 시 재시도 처리 복잡도가 높은 애플리케이션 단의 제어(낙관적 락)보다는 DBMS 차원에서 강력하게 정합성을 보장하는 것이 더 안정적이라고 판단했습니다.

이에 따라 ItemRepository의 아이템 조회 메서드에 @Lock(LockModeType.PESSIMISTIC_WRITE)를 적용하여 DB Row-level 쓰기 잠금을 걸었습니다. 추가로 무한 대기로 인한 데드락이나 병목 현상을 방지하기 위해 @QueryHint를 통해 lock.timeout(1000) (1초 대기)을 설정하여 안전장치를 마련했습니다.

결과 및 배운점

동시 다발적인 트래픽 상황에서도 트랜잭션을 직렬화하여 처리함으로써 데이터 정합성을 100% 보장하고 초과 결제율을 0%로 확실하게 차단했습니다. 데이터 무결성을 서버 로직에만 의존하지 않고 DBMS 레벨에서 원천 제어하는 락 매커니즘의 중요성과 견고한 아키텍처 설계의 필요성을 배웠습니다.

Lock 적용 전 테스트 결과

```
ItemPurchaseConcurrencyTest > 재고가 10개인 아이템을 100번 동시에 구매 요청할 경우, 10개만 판매되고 초과 결제가 발생하지 않아야 한다. STANDARD_ERROR
```

```
❌ [테스트 실패] - 초과 판매 발생
▶ 재고 : 10개 | 판매 : 90개 | 남은 재고 : -80개
▶ 100명의 동시 결제 요청 중 90건이 승인되었습니다.
```

```
ItemPurchaseConcurrencyTest > 재고가 10개인 아이템을 100번 동시에 구매 요청할 경우, 10개만 판매되고 초과 결제가 발생하지 않아야 한다. FAILED
```

비관적 락 코드

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@QueryHints({
    @QueryHint(name = "jakarta.persistence.lock.timeout", value = "1000")
})
```

Lock 적용 후 테스트 결과

```
ItemPurchaseConcurrencyTest > 재고가 10개인 아이템을 100번 동시에 구매 요청할 경우, 10개만 판매되고 초과 결제가 발생하지 않아야 한다. STANDARD_OUT
```

```
✅ [테스트 성공] - 동시성 제어 완료
▶ 재고 : 10개 | 판매 : 10개 | 남은 재고 : 0개
▶ 100명의 동시 결제 요청 중 10건이 승인되었습니다.
```

```
BUILD SUCCESSFUL in 37s
```


트러블 슈팅

Problem 2

컨트롤러 권한 검증의 중복 및 누락 문제

문제상황

학생용 API와 교사용 API가 혼재된 시스템에서 각 엔드포인트마다 접근 권한 인가(Authorization) 처리가 필수적이었습니다. 기존에는 모든 컨트롤러 메서드 최상단에 if (UserContext.get().getMemberRole() != MemberRole.TEACHER) throw new... 형태의 역할 검증 코드가 3~4줄씩 중복으로 삽입되었습니다. API가 50개 이상으로 늘어남에 따라 200줄 이상의 중복 코드가 발생했습니다. 비즈니스 로직과 권한 검증 로직이 강하게 결합되어 가독성과 유지보수성이 크게 떨어졌습니다.

해결과정

관심사 분리(Separation of Concerns)를 위해 AOP를 도입했습니다. 메서드 레벨에서 사용할 수 있는 커스텀 어노테이션(@TeacherOnly, @StudentOnly)을 정의하고, 이를 가로채는 RoleCheckAspect 클래스를 구현했습니다. @Before Advice를 사용해 컨트롤러의 비즈니스 로직이 실행되기 직전에 UserContext에서 역할을 확인하고, 권한이 없을 경우 즉각 예외를 던지도록 중앙 제어 계층을 신설했습니다.

결과 및 배운점

비즈니스 로직과 부가 기능(보안/권한 검증)을 완벽하게 분리하여 컨트롤러 코드를 매우 간결하게 유지할 수 있었습니다. 검증 방식이 변경되거나 새로운 역할(Role)이 추가되더라도 RoleCheckAspect 단일 클래스만 수정하면 시스템 전체에 즉시 반영되므로 유지보수 비용을 획기적으로 낮출 수 있었습니다. 공통 관심사를 AOP 로직으로 추출하는 것이 객체지향 설계에 얼마나 큰 이점을 주는지 깨달았습니다.

AOP 적용 전 코드

```
// 아이템 생성
@PostMapping
public ResponseEntity<ApiResponse<ItemResponseDto>> createItem(HttpServletRequest httpServletRequest,
    @RequestBody @Valid ItemDto request) {
    Member member = UserContext.get();
    if (member.getMemberRole() != MemberRole.TEACHER) {
        throw new BusinessException(ErrorCode.REQUEST_DENIED);
    }
    Integer classId = (Integer)httpServletRequest.getAttribute(CLASS_ID);
    validateClassId(classId);
    ItemResponseDto result = itemService.createItem(request, classId);
    return ResponseEntity.status(201).body(ApiResponse.success(result));
}
```

AOP 적용 후 코드

```
// 아이템 생성
@TeacherOnly
@PostMapping
public ResponseEntity<ApiResponse<ItemResponseDto>> createItem(HttpServletRequest httpServletRequest,
    @RequestBody @Valid ItemDto request) {

    Integer classId = (Integer)httpServletRequest.getAttribute(CLASS_ID);
    validateClassId(classId);
    ItemResponseDto result = itemService.createItem(request, classId);
    return ResponseEntity.status(201).body(ApiResponse.success(result));
}
```



프로젝트명

DOCSHUND 우수상 수상 🏆

개요

개발자를 위한 개발공식문서 번역 및 실시간 베스트 번역본 제공 서비스.
커뮤니티/채팅/챗봇 등의 번역 보조 툴 지원.

개발기간

2025.01.06 ~ 2025.02.21 (7주)

개발인원

6명

담당역할

프론트엔드, UI/UX디자인

결과물

[Github](#) [발표자료](#) [영상자료](#)

기술 환경

프론트엔드

React, Zustand, Tailwind CSS, Axios

백엔드

Spring Boot, Spring Security, JPA, QueryDSL

데이터베이스

MariaDB, MySQL

데브옵스

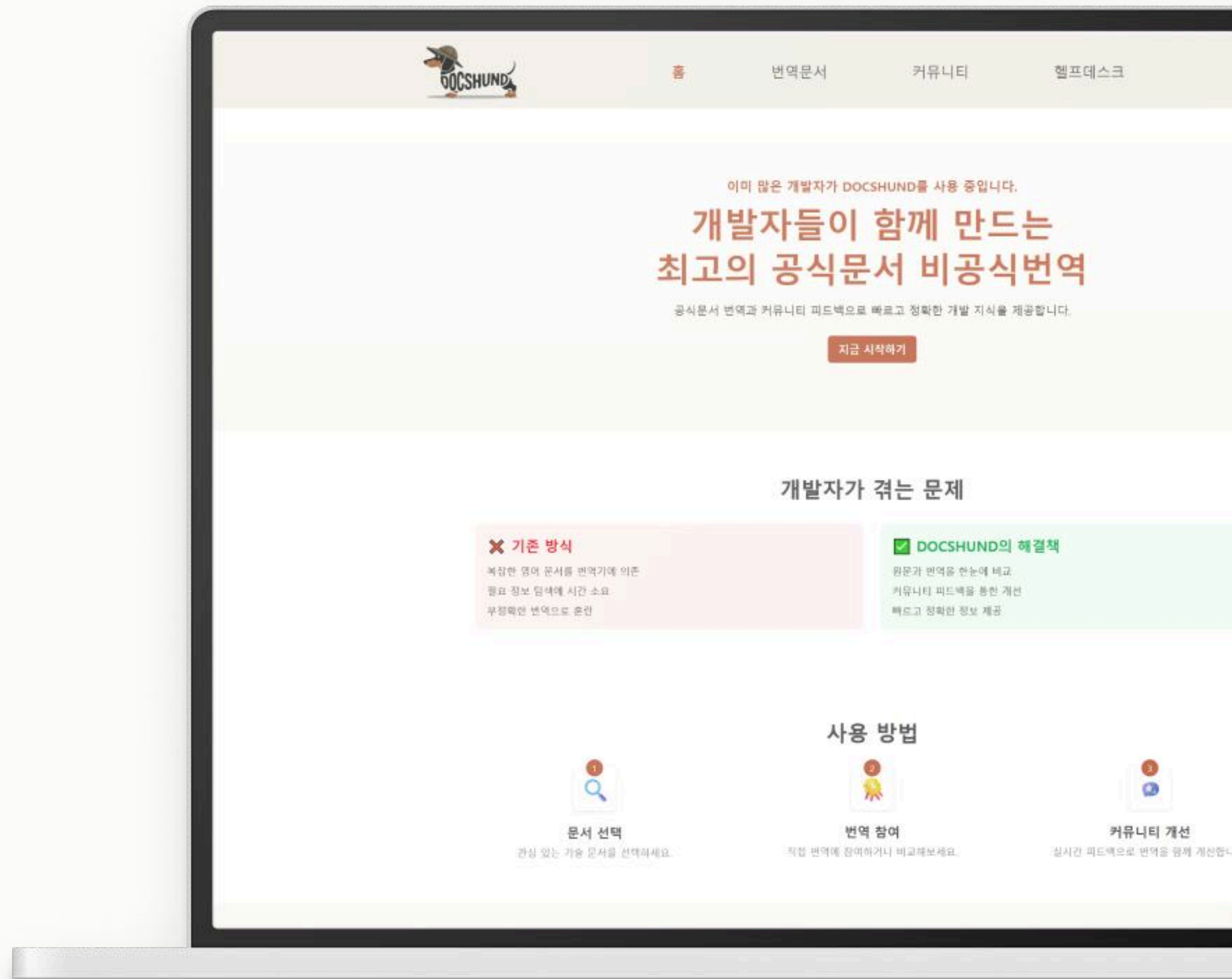
Jenkins, Docker, Nginx, AWS EC2

협업

Jira, Gitlab, Figma, Notion

기타

WebSocket, SSE, indexDB, JWT



개발 공식문서 번역 플랫폼 |

STOMP.js Pub/Sub 모델을 다루어 다채널 문서별 채팅방을 렌더링하고, Axios 전역 인터셉터와 라우팅 가드를 조합하여 보안 오류 0%에 수렴하는 무결점 프론트엔드 앱을 설계했습니다.

DocshunD





Problem 1

채팅 UX(스크롤 위치) 최적화 이슈

문제상황

기존 인터페이스는 새 메시지 수신 시 사용자의 시점과 상관없이 스크롤을 최하단으로 강제 이동시켰습니다. 이로 인해 활발한 대화 도중 과거 메시지를 읽고 있으면 화면이 수시로 아래로 튕겨 내려가 읽기 흐름이 자주 끊기는 심각한 불편함이 있었습니다. 또한, 과거 대화를 추가로 불러올 때 상단에 메시지가 삽입되면서 현재 보고 있던 텍스트 위치가 아래로 밀려나는 'UI 점핑' 현상이 발생해 서비스의 전반적인 완성도를 저해하고 있었습니다.

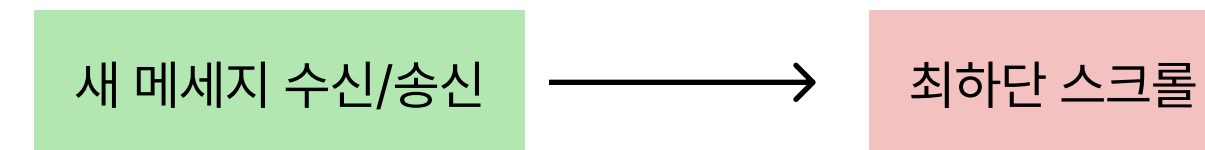
해결과정

사용자의 현재 위치와 발신 주체를 판단하는 조건부 스크롤 제어 로직을 구현했습니다. 먼저, 수신 메시지가 타인의 글일 경우 스크롤이 최하단 근처일 때만 자동으로 내려주고, 과거 대화 시청 중에는 위치를 고정해 읽기 환경을 보호했습니다. 반면, 본인이 보낸 메시지는 즉시 최하단으로 이동시켜 전송 성공을 바로 확인하게 했습니다. 특히 페이지네이션 시에는 데이터 추가 전후의 높이 차이를 계산해 스크롤 위치를 동적으로 보정함으로써 화면 튕 현상을 방지했습니다.

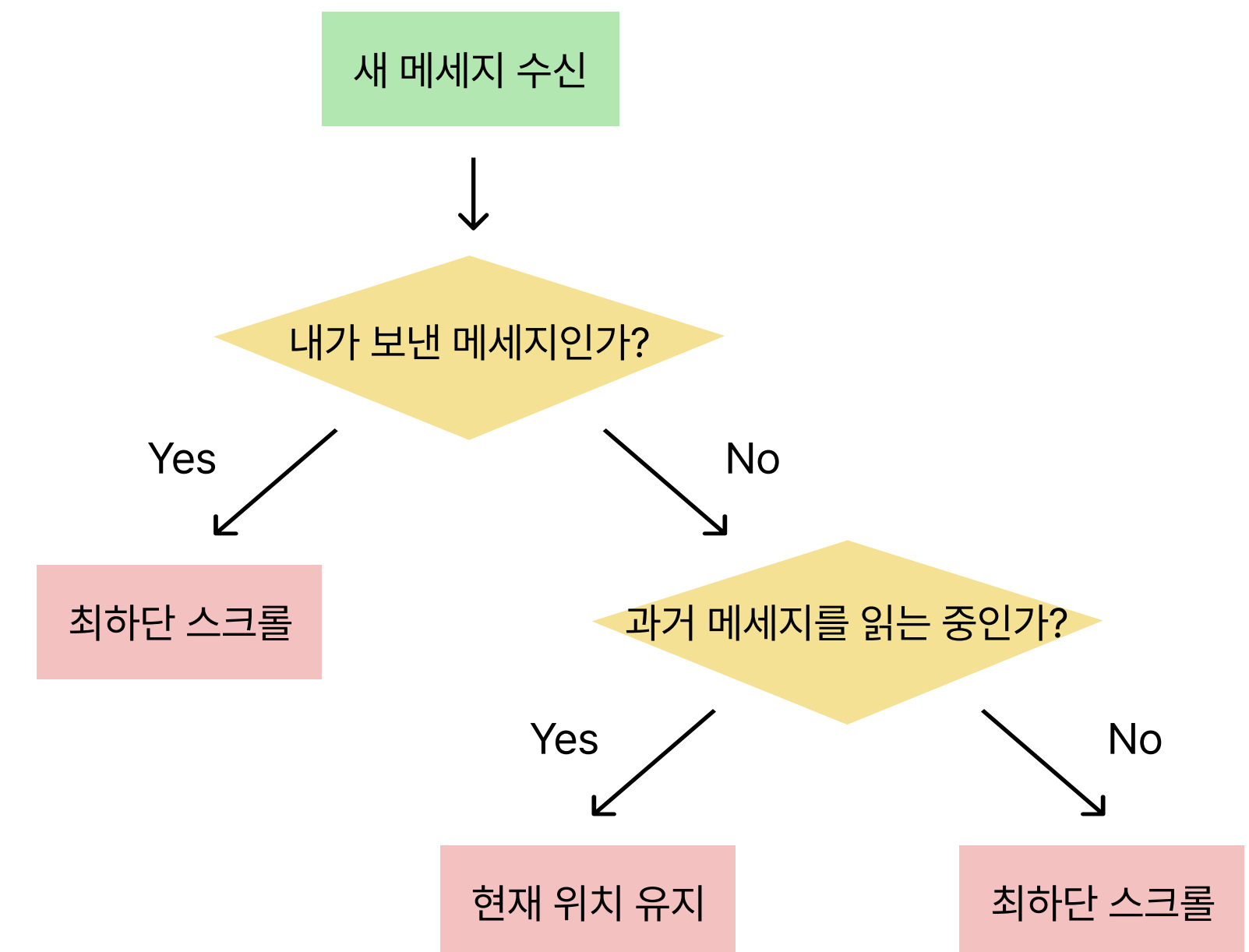
결과 및 배운점

결과적으로 상용 메신저와 유사한 수준의 안정적이고 매끄러운 채팅 UX를 완성했습니다. 대량의 메시지 유입 상황에서도 사용자는 방해 없이 대화를 탐색할 수 있게 되었으며, 서비스의 품질을 높이는 긍정적인 경험을 제공했습니다. 이번 과정을 통해 거창한 기능 구현보다 사용자의 작은 불편을 해결하는 '디테일'이 서비스의 완성도를 결정한다는 것을 깨달았습니다. 브라우저의 레이아웃 API를 정교하게 제어하며 UI 동작 원리에 대한 이해를 높였고, 사용자 중심의 사고방식이 코드에 어떻게 투영되어야 하는지 깊이 고민해볼 수 있었습니다.

UX FlowChart 변경 전



UX FlowChart 변경 후



트러블 슈팅

Problem 2

버튼 다중 클릭으로 인한 서버 과부하 및 500 에러 발생

문제상황

사용자가 처리 지연을 참지 못하거나 실수로 버튼을 연속해서 여러 번 누르는 '다중 클릭' 현상이 빈번하게 발생했습니다. 이로 인해 동일한 데이터 처리 요청(생성, 삭제, 좋아요 등)이 백엔드 서버로 단기간에 수십 번씩 보내지는 문제가 발생했습니다. 결과적으로 불필요한 API 호출이 폭주하면서 DB 커넥션 풀 고갈과 데드락을 유발하였고, 결국 서버가 부하를 견디지 못해 "HTTP 500 내부 서버 오류"를 반환하거나 불필요한 중복 데이터가 무더기로 생성되는 현상이 발생했습니다.

해결과정

무분별한 일괄 처리를 지양하고, 기능의 성격과 직관적인 UX를 고려하여 두 가지 전략으로 도입했습니다. 먼저, 게시글 좋아요나 댓글 등록처럼 사용자의 단순 클릭이 빈번한 요소에는 lodash.debounce(100~300ms)를 적용하여, 짧은 시간 내의 중복 클릭은 무시하고 마지막 1회만 서버로 전송되도록 처리했습니다. 반면, 글쓰기나 수식 제출처럼 데이터 트랙잭션이 무거운 로직은 Loading Lock(로딩 비활성화) 방식을 채택했습니다. 의도적인 지연 처리가 자칫 '응답 없음'으로 오인될 수 있기 때문에, 클릭 즉시 버튼을 disabled 처리하고 로딩 UI를 띄워 즉각적인 피드백을 제공함과 동시에 서버로의 중복 요청을 원천적으로 차단했습니다.

결과 및 배운점

사용자의 다발성 반복 클릭 이벤트로 인해 빈번히 발생하던 백엔드 500 에러 및 중복 데이터 생성 문제를 0%로 방지했습니다. 단순히 모든 버튼을 일괄 비활성화하거나 무조건 Debounce 처리를 거는 단편적인 해결책을 넘어, "기능의 목적과 사용자 인터페이스(UX)"에 맞춰 프론트엔드 레벨에서 적절한 방어 로직을 전략적으로 설계해야만 사용자 편의성을 살리면서 백엔드 서버의 안정성과 데이터 무결성을 동시에 지켜낼 수 있다는 점을 알게 되었습니다.

개발 공식문서 번역 플랫폼 |



Debounce 적용 전

```
[Before] API 호출 시도됨! #1 (2:38:53 PM)
[Before] API 호출 시도됨! #2 (2:38:53 PM)
[Before] API 호출 시도됨! #3 (2:38:53 PM)
[Before] API 호출 시도됨! #4 (2:38:54 PM)
[Before] API 호출 시도됨! #5 (2:38:54 PM)
[Before] API 호출 시도됨! #6 (2:38:54 PM)
[Before] API 호출 시도됨! #7 (2:38:54 PM)
[Before] API 호출 시도됨! #8 (2:38:54 PM)
[Before] API 호출 시도됨! #9 (2:38:54 PM)
[Before] API 호출 시도됨! #10 (2:38:55 PM)
[Before] API 호출 시도됨! #11 (2:38:55 PM)
[Before] API 호출 시도됨! #12 (2:38:55 PM)
[Before] API 호출 시도됨! #13 (2:38:55 PM)
[Before] API 호출 시도됨! #14 (2:38:55 PM)
[Before] API 호출 시도됨! #15 (2:38:55 PM)
[Before] API 호출 시도됨! #16 (2:38:56 PM)
[Before] API 호출 시도됨! #17 (2:38:56 PM)
[Before] API 호출 시도됨! #18 (2:38:56 PM)
[Before] API 호출 시도됨! #19 (2:38:56 PM)
[Before] API 호출 시도됨! #20 (2:38:56 PM)
[Before] API 호출 시도됨! #21 (2:38:56 PM)
[Before] API 호출 시도됨! #22 (2:38:57 PM)
[Before] API 호출 시도됨! #23 (2:38:57 PM)
[Before] API 호출 시도됨! #24 (2:38:57 PM)
```

Debounce 적용 후

```
[After] 버튼 클릭됨... (Wait 500ms...) #1
[After] 버튼 클릭됨... (Wait 500ms...) #2
[After] 버튼 클릭됨... (Wait 500ms...) #3
[After] 버튼 클릭됨... (Wait 500ms...) #4
[After] 버튼 클릭됨... (Wait 500ms...) #5
[After] 버튼 클릭됨... (Wait 500ms...) #6
[After] 버튼 클릭됨... (Wait 500ms...) #7
[After] 버튼 클릭됨... (Wait 500ms...) #8
[After] 버튼 클릭됨... (Wait 500ms...) #9
[After] 버튼 클릭됨... (Wait 500ms...) #10
[After] 버튼 클릭됨... (Wait 500ms...) #11
[After] 버튼 클릭됨... (Wait 500ms...) #12
[After] 버튼 클릭됨... (Wait 500ms...) #13
[After] 버튼 클릭됨... (Wait 500ms...) #14
[After] 버튼 클릭됨... (Wait 500ms...) #15
[After] 버튼 클릭됨... (Wait 500ms...) #16
[After] 버튼 클릭됨... (Wait 500ms...) #17
[After] 버튼 클릭됨... (Wait 500ms...) #18
[After] 버튼 클릭됨... (Wait 500ms...) #19
[After] 버튼 클릭됨... (Wait 500ms...) #20
[After] 버튼 클릭됨... (Wait 500ms...) #21
[After] 버튼 클릭됨... (Wait 500ms...) #22
[After] 최종 API 호출 실행!!! (총 22번 연타 중 마지막 1번만)
```

Loading UI

제목 *

새로운 공식 문서 번역 업데이트 요청드립니다.

25 / 80

이메일 *

이메일을 잘못 입력한 경우, 메일이 전송되지 않을 수 있습니다.

reactuser@mail.com

내용 *

안녕하세요!
항상 DOCSHUND 서비스 유용하게 잘 사용하고 있습니다. **

다름이 아니라 이번에 새로 나온 React 버전이 대해 공식 문서 번역 컨텐츠가 추가되었으면 좋겠어서 문의 남깁니다.
시간 되실 때 검토 부탁드립니다. 좋은 하루 되세요!

144 / 2000

사진 첨부 (1개만 가능)

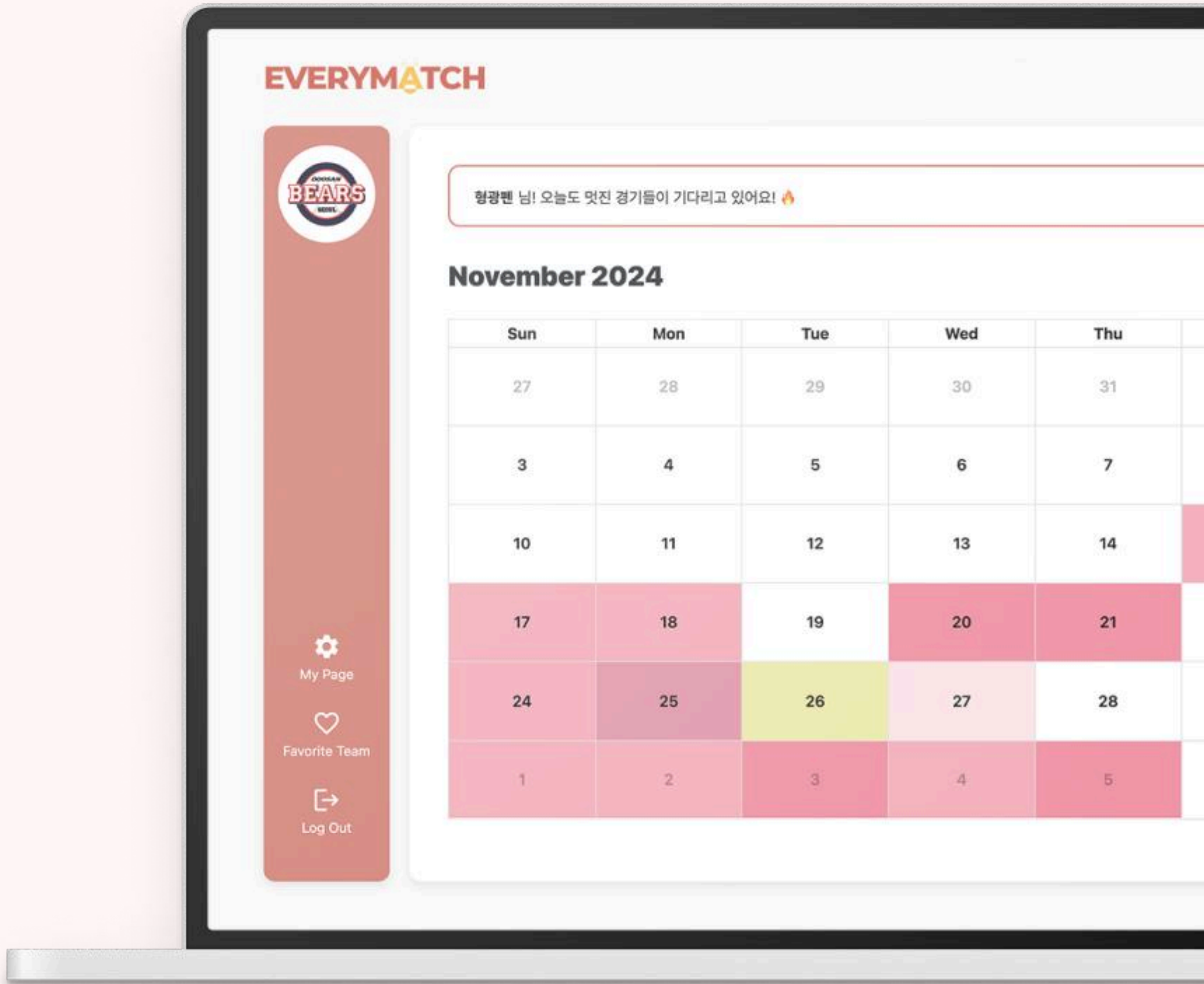
사진 선택

첨부할 사진을 선택하세요

보내기

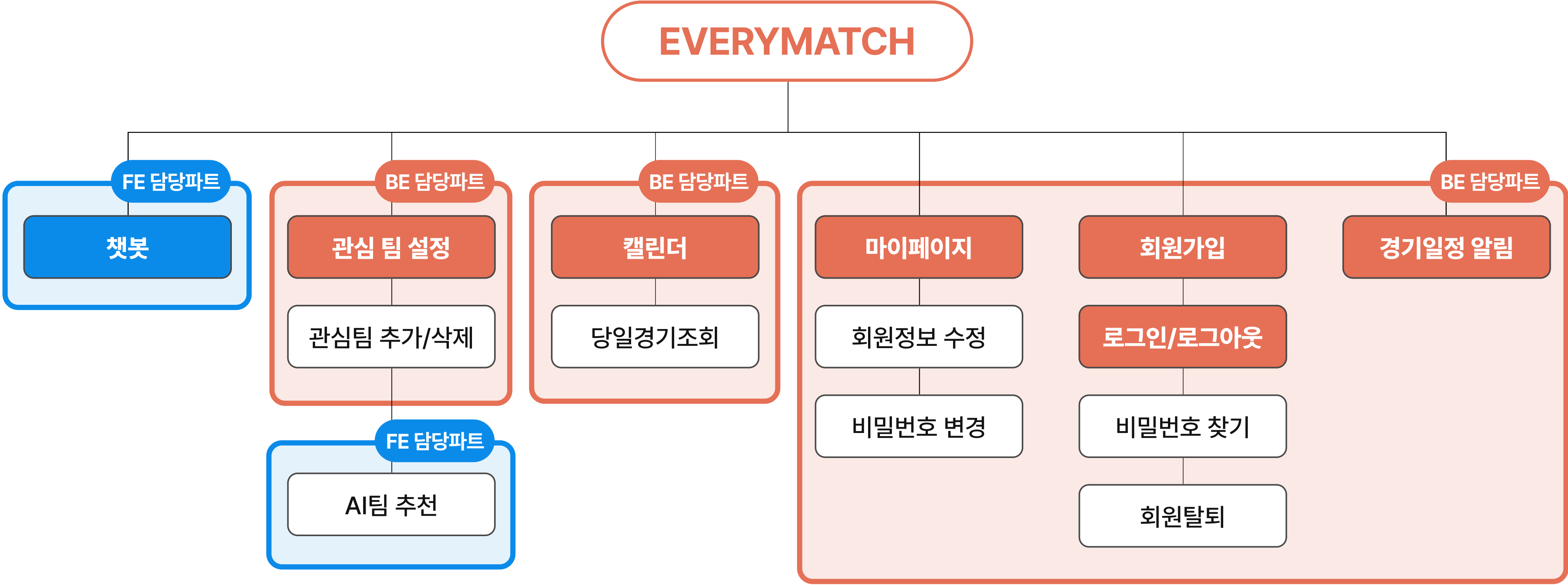


프로젝트명	EVERYMATCH 우수상 수상 🏆
소개	다양한 스포츠 팀의 경기 일정을 한눈에! 관심 팀을 선택하고 통합으로 관리할 수 있는 캘린더
개발기간	2024.11.18 ~ 2024.11.27 (10일)
개발인원	2명
담당역할	백엔드, 프론트엔드, UI/UX디자인
결과물	 Github  발표자료
기술 환경	
프론트엔드	Vue.js, Pinia, CSS, Vue Router, Axios
백엔드	Spring Boot, MyBatis
데이터베이스	MySQL
협업	Github, Figma, Notion
기타	JWT, JBCrypt



| 경기 일정관리 웹 서비스

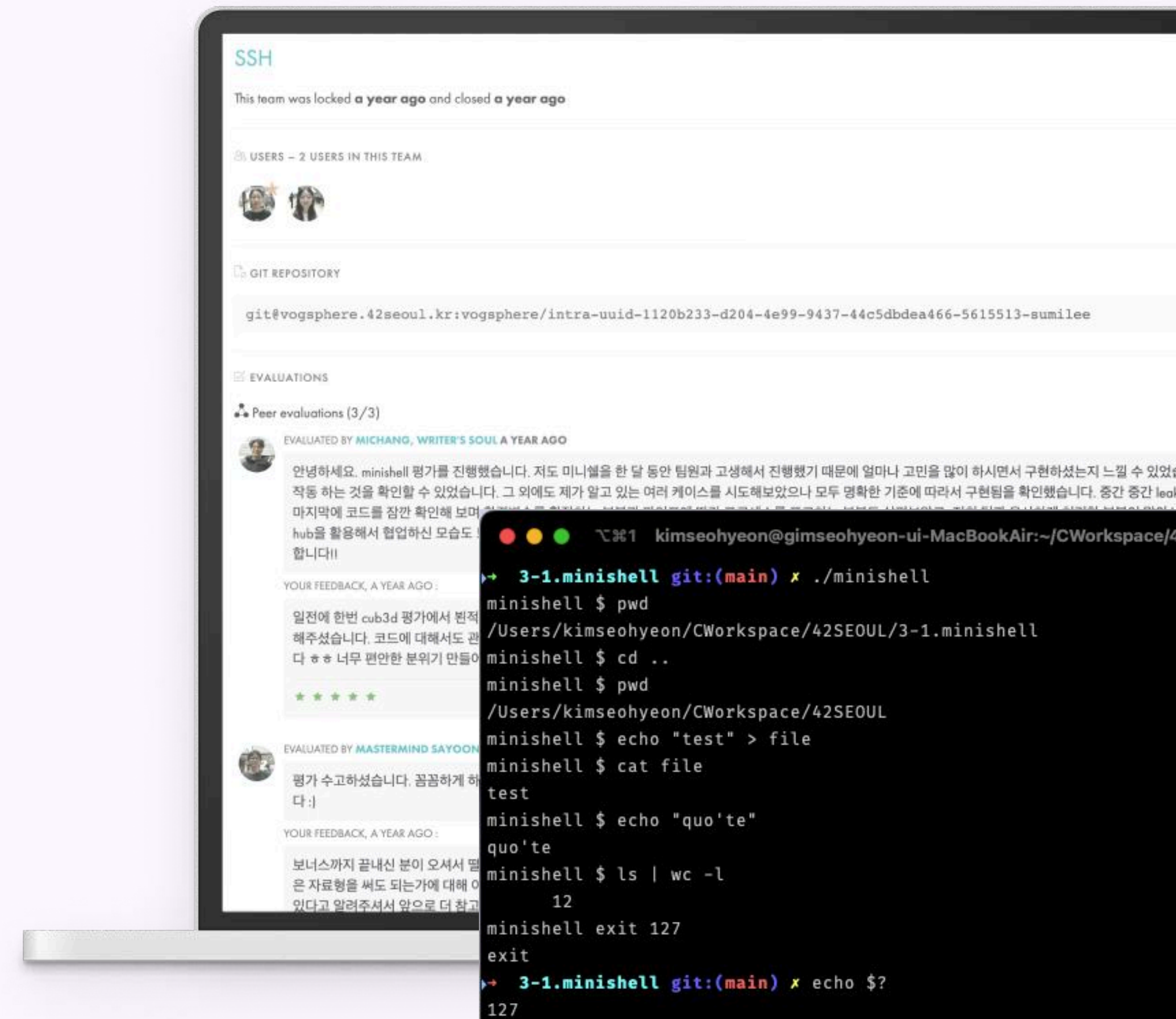
10일 이라는 짧은 풀스택 개발 기간 동안, Spring Security/JWT 기반의 견고한 인증부터 Spring Scheduler 기반 이메일 알람 배치 자동화까지 기능의 깊이와 완성도를 증명해 냈습니다.



프로젝트명	Minishell
소개	Bash Shell과 유사하게 동작하는 나만의 쉘 만들기
개발기간	2024.03.09 ~ 2024.04.01 (3주)
개발인원	2명
담당역할	명령어 파싱, 시그널 처리
결과물	 Github

주요 기능 * 강조된 부분 담당 개발 파트

- 입력된 명령어 파싱
- 닫히지 않은 따옴표나 특정되지 않은 특수문자(\, ;)를 해석하지 않음
- -n 옵션을 사용할 수 있는 echo
- 상대, 절대경로만 사용하는 cd
- 옵션없는 pwd
- 옵션이 없는 exit
- 리다이렉션 구현
- 파이프(|) 구현
- ctrl + C, ctrl + D, ctrl + \ 입력시 bash shell과 동일하게 동작



| BashShell 클론코딩

10일 이라는 짧은 풀스택 개발 기간 동안, Spring Security/JWT 기반의 견고한 인증부터 Spring Scheduler 기반 이메일 알람 배치 자동화까지 기능의 깊이와 완성도를 증명해 냈습니다.

Appendix

기술적 성장 과정

클린코드

단순히 기능을 만드는 데 급급했던 습관을 버리고, 제약 사항(Norminette)을 논리적 모듈화의 기회로 삼았습니다.

- 상황 : 42서울 과제에는 함수 25줄 제한, 매개변수 4개 이하, 한 파일당 함수 5개 등 극한의 제약이 있었습니다. 제약 속에서 비대해진 로직을 최소 단위로 분리하고 재사용성을 고민하며 추상화 능력을 길렀습니다.
- 결과 : 제약을 '방해'가 아닌 '설계 가이드'로 받아들이며, 누가 읽어도 구조가 한눈에 들어오는 가독성 높은 코드를 작성하게 되었습니다.

개발태도

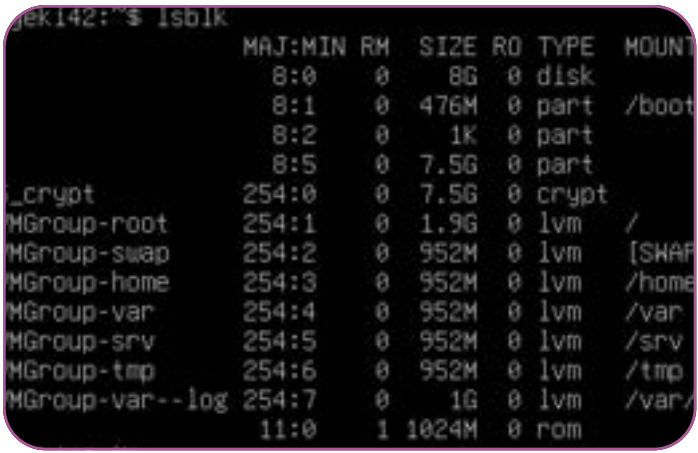
랜덤 동료 평가 시스템을 통해 본인의 코드를 객관적으로 바라보고, 기술적 선택에 책임을 지는 태도를 길렀습니다.

- 상황 : 내가 작성한 모든 코드에 대해 기술적 근거를 논리적으로 설명하는 훈련을 반복했습니다. 쏟아지는 질문 속에서 제 로직의 허점을 발견하고 수정하며 코드의 완성도를 높였습니다.
- 결과 : 이 과정에서 자신의 로직을 논리적으로 디펜스하는 커뮤니케이션 역량을 얻었으며, 동료의 피드백을 성장의 밑거름으로 삼는 유연함을 갖추게 되었습니다.

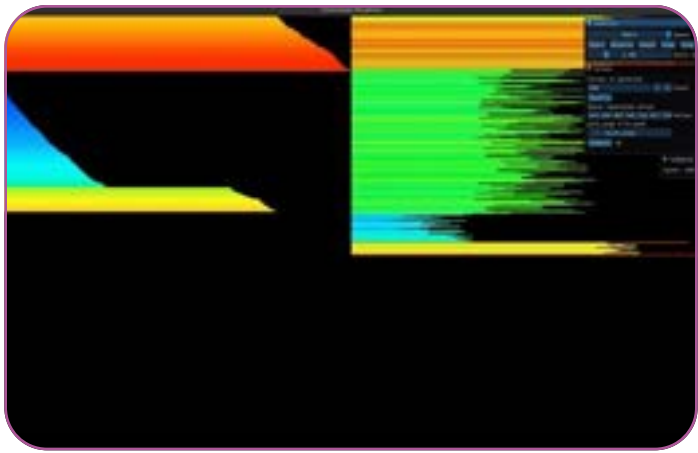
완성도

메모리 누수(Memory Leak)를 0으로 만드는 철저한 관리 경험을 통해, 안정적인 소프트웨어를 만드는 엔지니어링 마인드를 확립했습니다.

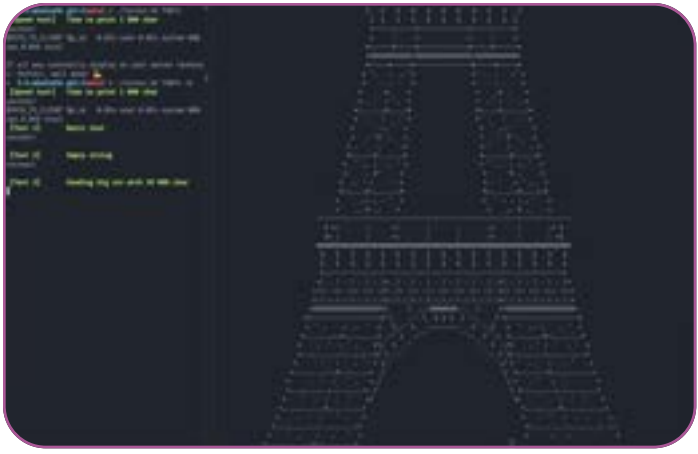
- 상황 : 동적 할당 하나에도 메모리 해제 시점을 집요하게 추적하며, 발생 가능한 모든 엣지 케이스를 사전에 차단하는 방어적 프로그래밍의 중요성을 깨달았습니다.
- 결과 : 이 경험은 어떤 환경에서도 신뢰할 수 있는 서비스를 만드는 견고한 개발 철학의 밑거름이 되었습니다.



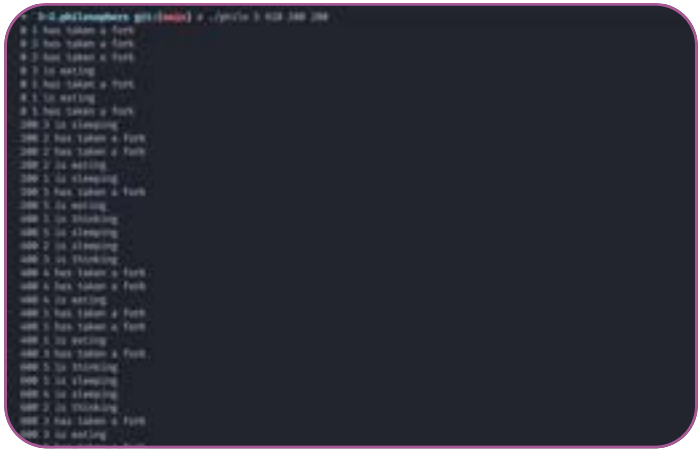
Born2BeRoot
virtualbox를 활용하여,
CLI환경에서 OS설치, ufw, ssh, sudo설정,
monitoring.sh작성 등 환경 세팅하기



PushSwap
스택A와 스택B 2개를 가지고 명령어를 최대한 적게 사용하여 정렬하기



Minitalk
signal을 이용해 데이터를 주고받는 작은 프로그램 작성하기



Philosophers
pthread.h를 사용하여 멀티 쓰레드환경을 만들고, mutex로 decdlock을 해결하기



Cub3D
레이캐스팅을 이용하여 1인칭 시점의 미로를 3D로 구현하기



So_Long
map의 유효성을 확인하고, mlx라이브러리를 사용하여 간단한 2D게임 만들기.